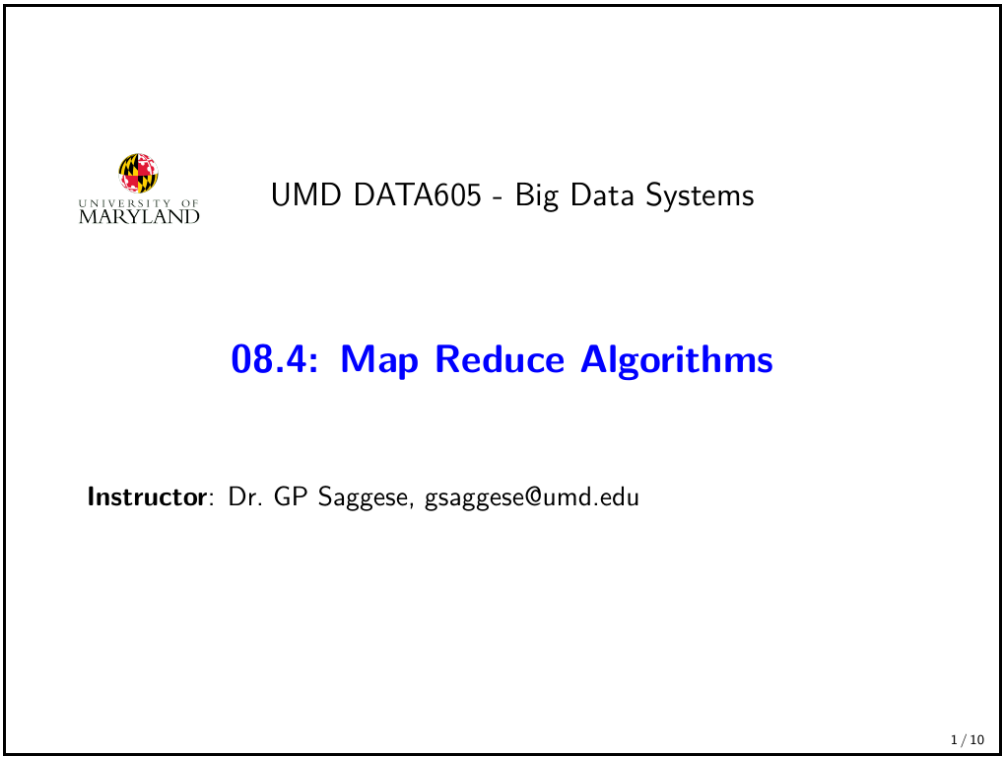
08.4: Map Reduce Algorithms



2 / 10: MapReduce: Applications

MapReduce: Applications

- **Intuition**
  - Break massive jobs into independent map tasks
  - Aggregate via shuffle
  - Combine in reduce
- **Major classes of applications**
  - Text processing and search
    - E.g., tokenization, inverted index, log analysis
  - Large data transforms
    - E.g., ETL pipelines, joins, global sort, deduplication across petabytes
  - Data mining and machine learning
    - E.g., Counting co occurrences, feature extraction, k-means iterations
  - Graph and link analysis
    - E.g., PageRank, connected components
- **Typical outputs**, e.g.,
  - Counts
  - Aggregates
  - Reorganized datasets for downstream systems

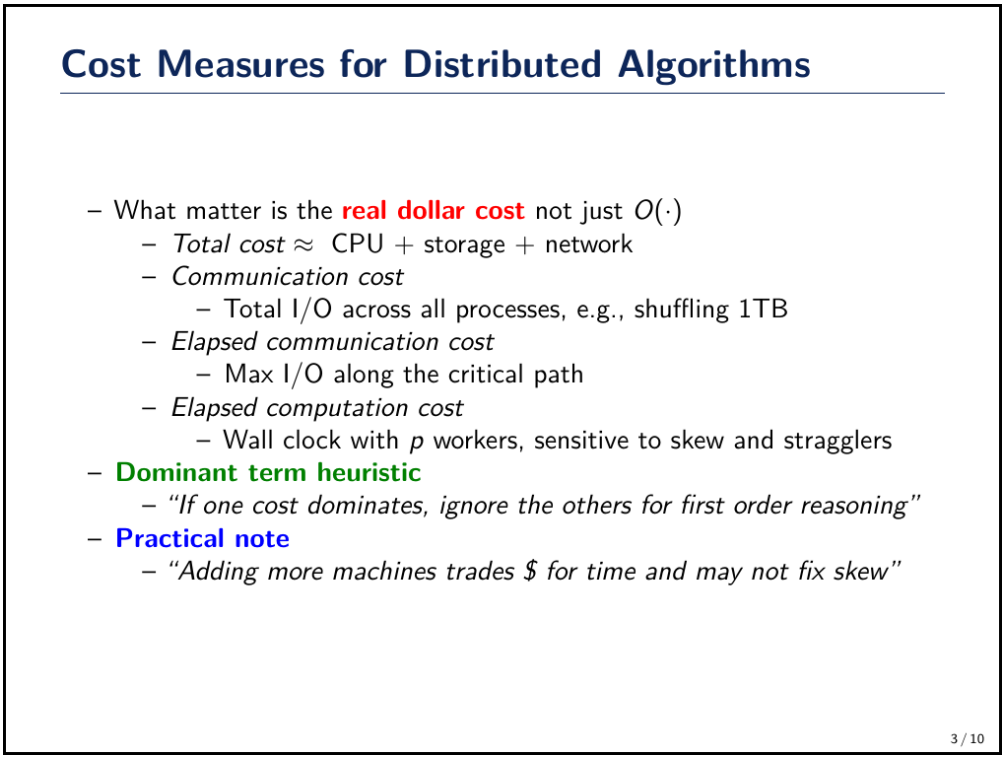
2 / 10

- **MapReduce: Applications**
- **Intuition**
  - *Break massive jobs into independent map tasks*: The core idea of MapReduce is to divide a large problem into smaller, manageable pieces. Each piece, or "map task," can be processed independently, which makes it easier to handle large datasets.
  - *Aggregate via shuffle*: After the map tasks are completed, the results are shuffled. This means that the data is reorganized so that all related pieces are grouped together, which is crucial for the next step.
  - *Combine in reduce*: Finally, the grouped data is processed by "reduce tasks" that combine the results to produce the final output. This step is where the actual aggregation or summarization happens.
- **Major classes of applications**
  - *Text processing and search*: This includes tasks like breaking down text into words (tokenization), creating an index for fast search (inverted index), and analyzing logs to extract useful information.
  - *Large data transforms*: These are operations that change the structure or format of data, such as extracting, transforming, and loading data (ETL), joining datasets, sorting data globally, or removing duplicates from very large datasets.
  - *Data mining and machine learning*: MapReduce can be used for tasks like counting how often things occur together (co-occurrences), extracting features for machine learning models, or running iterative algorithms like k-means clustering.
  - *Graph and link analysis*: This involves analyzing relationships and connections, such

as calculating the importance of web pages (PageRank) or finding groups of connected nodes (connected components).

- **Typical outputs**, e.g.,
  - *Counts*: Simple tallies of occurrences, such as word counts in a document.
  - *Aggregates*: Summarized data, like averages or totals, that provide insights into the dataset.
  - *Reorganized datasets for downstream systems*: Data that has been processed and structured in a way that makes it ready for further analysis or use in other systems.

3 / 10: Cost Measures for Distributed Algorithms



- **Cost Measures for Distributed Algorithms**
  - When evaluating distributed algorithms, it's crucial to consider the *real dollar cost* rather than just the theoretical complexity, often denoted as $O(\cdot)$. This means looking at the actual expenses involved in running the algorithm.
  - **Total cost** is a combination of three main factors: CPU usage, storage requirements, and network communication. Each of these contributes to the overall expense of executing a distributed algorithm.
  - **Communication cost** refers to the total input/output operations across all processes. For example, if you need to shuffle 1TB of data between machines, this is a significant part of the cost.
  - **Elapsed communication cost** is the maximum input/output along the critical path, which is the sequence of dependent tasks that determine the total execution time.
  - **Elapsed computation cost** is the actual time taken to complete the computation with p workers. This can be affected by *skew* (uneven distribution of work) and *stragglers*

(slow tasks that delay the overall process).

- **Dominant term heuristic**
  - This principle suggests that if one cost (e.g., communication or computation) is significantly larger than the others, you can focus on that cost for initial analysis. This simplifies reasoning about the algorithm's efficiency.
- **Practical note**
  - Adding more machines can reduce the time taken to complete tasks by distributing the workload. However, this comes at a financial cost and may not solve issues like skew, where some tasks take longer due to uneven distribution of work.

4 / 10: Total Cost Model for MapReduce

Total Cost Model for MapReduce

- **Total cost** of a computation

$$C_{total} = C_{compute} + C_{io} + C_{network} + C_{storage}$$

- **Notation**
  - $|I|$ input GB, $|S|$ shuffle GB, $|O|$ output GB
  - p_m mappers, p_r reducers
  - T_m map hours, T_r reduce hours
  - $c_{compute}$ = \$/VM hour, c_{io} = \$/GB I/O, $c_{shuffle}$ = \$/GB shuffle
- **Compute cost**

$$C_{compute} = c_{compute}\,(p_m T_m + p_r T_r)$$

where T_m, T_r include skew and stragglers

  - Skew effects = a heavy key or hotspot inflates the heaviest task and the critical path
- **I/O cost**

$$C_{io} = c_{io}\,(|I| + 2|S| + |O|)$$

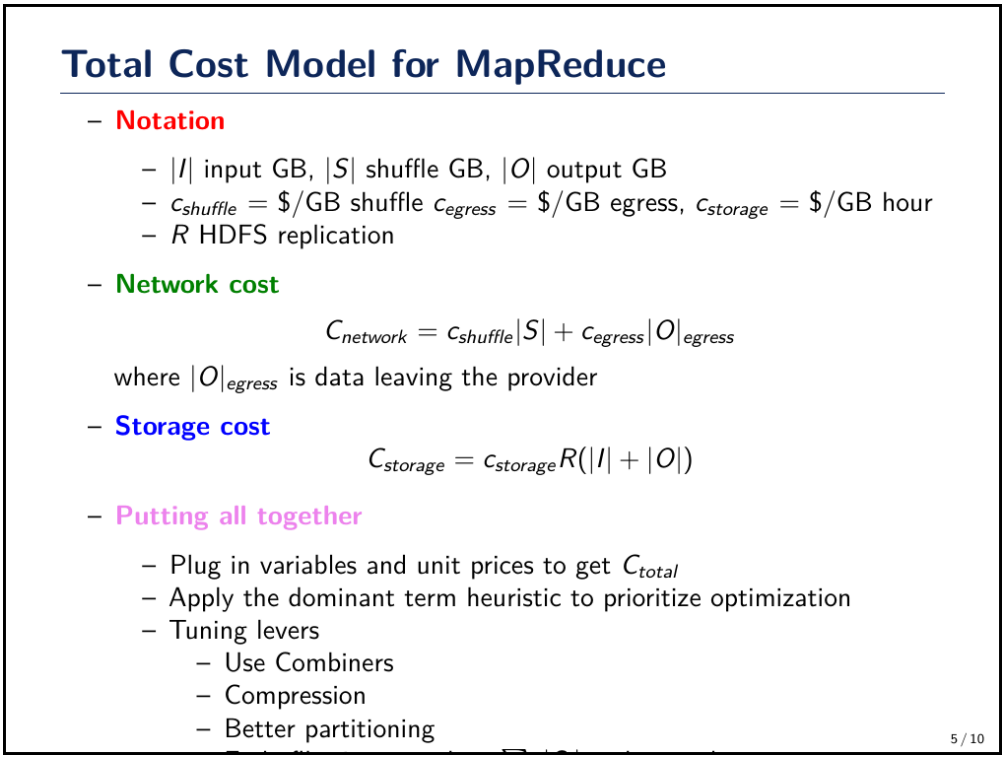
  - Shuffle often dominates since $\sum_i |S_i| \gg |I|, |O|$

4 / 10

- **Total cost of a computation**
  - The total cost of running a MapReduce job is a combination of several factors. These include the cost of computation, input/output operations, network data transfer, and storage. The formula provided sums up these components to give the overall cost. Understanding this helps in optimizing and budgeting for large-scale data processing tasks.
- **Notation**
  - *Input, Shuffle, and Output*: $|I|$, $|S|$, and $|O|$ represent the sizes in gigabytes of the input data, the data shuffled between mappers and reducers, and the output data, respectively.
  - *Mappers and Reducers*: p_m and p_r denote the number of mappers and reducers used in the job.
  - *Time*: T_m and T_r are the hours spent on mapping and reducing tasks.
  - *Cost per unit*: $c_{compute}$ is the cost per virtual machine hour, c_{io} is the cost per gigabyte of I/O, and $c_{shuffle}$ is the cost per gigabyte of data shuffled.
- **Compute cost**

  - The compute cost is calculated by multiplying the cost per VM hour by the total hours spent on mapping and reducing tasks. This includes time lost due to skew and stragglers, which are inefficiencies caused by uneven data distribution or slow tasks that delay the entire job.
- **I/O cost**
  - The I/O cost accounts for the data read and written during the job. It includes the input data, twice the shuffle data (since it is read and written), and the output data. Shuffle operations often dominate the I/O cost because the amount of data shuffled is typically much larger than the input or output data sizes. Understanding this can help in optimizing data flow to reduce costs.

5 / 10: Total Cost Model for MapReduce



- **Total Cost Model for MapReduce**
  - **Notation**
    - *Input, Shuffle, and Output Data*: We use $|I|$ to represent the size of input data in gigabytes (GB), $|S|$ for the shuffle data size, and $|O|$ for the output data size.
    - *Cost Parameters*: $c_{shuffle}$ is the cost per GB for shuffling data, c_{egress} is the cost per GB for data leaving the provider, and $c_{storage}$ is the cost per GB per hour for storage.
    - *HDFS Replication*: R represents the replication factor in Hadoop Distributed File System (HDFS), which affects storage costs.
- **Network Cost**
  - The formula $C_{network} = c_{shuffle}|S| + c_{egress}|O|_{egress}$ calculates the network cost. Here, $|O|_{egress}$ is the portion of output data that exits the cloud provider, incurring additional egress costs.

- **Storage Cost**
  - The storage cost is given by $C_{storage} = c_{storage} R(|I| + |O|)$. This accounts for the cost of storing both input and output data, considering the replication factor R which multiplies the storage requirement.
- **Putting all together**
  - To find the total cost, substitute the specific values and prices into the formulas to compute C_{total}.
  - Use the *dominant term heuristic* to identify which cost component is the largest and focus optimization efforts there.
  - **Tuning Levers**: These are strategies to reduce costs:
    - *Use Combiners*: Minimize data transfer by combining data before shuffling.
    - *Compression*: Reduce data size to lower storage and network costs.
    - *Better Partitioning*: Optimize data distribution to improve processing efficiency.
    - *Early Filtering*: Reduce the amount of shuffle data and avoid delays caused by slow tasks (stragglers).

6 / 10: Inverted Index using MapReduce

Inverted Index using MapReduce

- **Goal**: build a mapping from words to the list of documents they appear in
- **Example**
  - Input in `doc1` = `MapReduce is powerful`
  - Output = `[(MapReduce, doc1), (is, doc1), (powerful, doc1)]`
  - *Map phase*:
    - Input: `(docID, content)`
    - Emit: `(word, docID)` for each word in content
  - *Reduce phase*:
    - Input: `(word, [docID_1, docID_2, ...])`
    - Emit: `(word, list of unique docIDs)`
- **Implementation considerations**
  - Useful in search engines and information retrieval
  - Requires tokenization and normalization of content
  - Deduplication of document IDs in reducer

6 / 10

- **Goal**: The main aim here is to create an *inverted index*, which is a data structure that maps each word to the list of documents where it appears. This is crucial for tasks like search engines, where you need to quickly find documents containing specific words.
- **Example**:
  - Imagine you have a document, `doc1`, with the text "MapReduce is powerful". The output of the process will be a list of pairs showing each word and the document it appears in, like `[(MapReduce, doc1), (is, doc1), (powerful, doc1)]`.

- **Map phase**:
  - In this phase, the input is a pair consisting of a document ID and its content. For each word in the content, the map function emits a pair of the word and the document ID. This means if a word appears multiple times in a document, it will be emitted multiple times with the same document ID.
- **Reduce phase**:
  - The reducer takes each word and a list of document IDs where the word appears. It processes this list to emit a pair of the word and a list of unique document IDs. This step is crucial to ensure that each document ID appears only once per word.
- **Implementation considerations**:
  - This technique is particularly useful in search engines and information retrieval systems, where quick access to documents containing specific words is needed.
  - Before processing, the content must be tokenized (split into words) and normalized (standardized, e.g., converting to lowercase).
  - During the reduce phase, it's important to remove duplicate document IDs to ensure the list of documents is unique for each word.

7 / 10: Join Operations using MapReduce

Join Operations using MapReduce

- **Goal**: join two datasets based on a common key
  - Types of joins:
    - Inner join, left/right outer join, full outer join
- **Example**
  - Join `EmployeeRecords(empID, name, deptID)` with `Department(deptID, deptName)`
  - Join key: `deptID`
  - *Map phase*:
    - Emit key as `deptID`, tag each record with source label (e.g., "E" or "D")
    - Example output: `(deptID, ("E", empRecord))`, `(deptID, ("D", deptRecord))`
  - *Reduce phase*:
    - Input: `(deptID, [list of tagged records])`
    - Cross-product logic based on tags to perform the actual join
- **Implementation considerations**
  - Use composite values with source tag to distinguish record origin
  - Optimize data layout to minimize network shuffle
  - Ensure proper partitioning so all records with the same key go to the same reducer

7 / 10

- **Join Operations using MapReduce**
  - **Goal**: The main objective here is to combine two datasets using a common key. This is a common task in data processing where you want to merge information from different sources. The types of joins you can perform include:
    - **Inner join**: Only includes records with matching keys in both datasets.

    - **Left/right outer join**: Includes all records from one dataset and the matched records from the other dataset. If no match is found, the result is NULL on the side that doesn't have a match.
    - **Full outer join**: Includes all records when there is a match in either dataset. If there is no match, the result is NULL for the missing side.
- **Example**
    - Imagine you have two datasets: `EmployeeRecords` with fields like `empID`, `name`, and `deptID`, and `Department` with fields like `deptID` and `deptName`. You want to join these datasets on the `deptID` field.
    - **Map phase**: This is the first step where each record is processed to emit a key-value pair. The key is the `deptID`, and the value is the record tagged with its source (e.g., "E" for EmployeeRecords and "D" for Department). This tagging helps in identifying the origin of the record during the reduce phase.
        - Example output might look like `(deptID, ("E", empRecord))` for an employee record and `(deptID, ("D", deptRecord))` for a department record.
    - **Reduce phase**: In this step, all records with the same `deptID` are brought together. The reducer receives a key and a list of tagged records. It then applies logic to combine these records based on their tags, effectively performing the join operation.
- **Implementation considerations**
    - It's important to use composite values with a source tag to keep track of where each record comes from. This helps in correctly merging the records during the reduce phase.
    - Optimizing data layout is crucial to minimize the amount of data that needs to be shuffled across the network, which can be a costly operation.
    - Proper partitioning is essential to ensure that all records with the same key end up at the same reducer. This is necessary for the join operation to work correctly, as it relies on having all related records together.

8 / 10: Sorting and Grouping in MapReduce

Sorting and Grouping in MapReduce

- **Goal**: organize data by keys or values for further analysis
- **Example**
  - Sort sales data by date or group by product ID
  - *Map phase*:
    - Emit data with key as sort/group criterion
  - *Shuffle phase*:
    - Shuffle and sort phase automatically sorts by key
  - *Reduce phase*:
    - Receives sorted keys and can perform grouped aggregation
- Often used as a preprocessing step for reporting

8 / 10

- **@Goal@**: The main aim of sorting and grouping in MapReduce is to organize data based on specific keys or values. This organization is crucial for performing further analysis efficiently. By sorting or grouping data, we can easily identify patterns, trends, or anomalies, which are essential for making informed decisions.
- **@Example@**:
  - Consider a scenario where you have sales data. You might want to sort this data by date to analyze sales trends over time or group it by product ID to see which products are selling the most.
  - ***Map phase***: In this phase, the data is processed, and each piece of data is emitted with a key that serves as the criterion for sorting or grouping. For example, the key could be the date or product ID.
  - ***Shuffle phase***: This phase is automatic in MapReduce. It takes care of sorting the data by the key. This means that all data with the same key will be grouped together.
  - ***Reduce phase***: Here, the sorted keys are received, and any necessary aggregation or further processing is performed. This could involve summing sales for each product or calculating averages.
- **Often used as a preprocessing step for reporting**: Sorting and grouping are often preliminary steps in data processing. They prepare the data for more detailed analysis or reporting, making it easier to generate insights and summaries that are useful for business intelligence or decision-making.

9 / 10: Graph Processing with MapReduce

Graph Processing with MapReduce

- Many graph algorithms are iterative
  - Requires multiple MapReduce rounds for convergence
  - Each iteration refines the scores
- PageRank:
  - Compute importance score of web pages
  - Graph is represented as adjacency lists
  - *Map phase*:
    - Emit contributions to neighboring nodes
  - *Reduce phase*:
    - Sum contributions to update PageRank score

9 / 10

- **Graph Processing with MapReduce**
  - **Many graph algorithms are iterative**
    - Graph algorithms often need to repeat their processes several times to get to a final answer. This is because they refine their results step by step, getting closer to the correct answer with each pass.
    - In the context of MapReduce, this means running multiple rounds of the MapReduce process. Each round helps to improve the accuracy of the results until they reach a stable state, known as convergence.
    - Each iteration is crucial because it helps in refining the scores or values that are being calculated, ensuring that the final output is as accurate as possible.
- **PageRank:**
  - PageRank is a well-known algorithm used to determine the importance of web pages. It assigns a score to each page based on the number and quality of links to it, which helps in ranking pages in search engine results.
  - The web is represented as a graph where each page is a node, and links between pages are edges. This graph is expressed using adjacency lists, which show which pages link to which other pages.
  - *Map phase*:
    - During the Map phase, the algorithm calculates how much "importance" each page should pass on to its neighbors. It does this by emitting contributions, which are essentially pieces of the PageRank score, to each of the pages it links to.
  - *Reduce phase*:
    - In the Reduce phase, all the contributions received by a page are summed up to update its PageRank score. This new score reflects the page's importance based on

the current iteration's data.

10 / 10: Statistical Aggregation and Log Analysis

Statistical Aggregation and Log Analysis

- **Statistical Aggregation**
  - Example: `(sensorID, temperature)` → compute average temperature per sensor
  - *Map phase*: emit relevant quantities (e.g., value, 1 for count)
  - *Reduce phase*: compute sums, averages, variances
- **Log Analysis**
  - Example: `(status code, 1)` → count occurrences
  - *Map phase*: parse logs, extract fields (timestamp, IP, status) and emit `(key, value)` for desired metrics
  - *Reduce phase*: aggregate (e.g., count errors, hits per IP)
- **Implementation considerations**
  - Useful for monitoring, alerting, and trend analysis
  - Can handle large-scale logs from distributed systems

10 / 10

- **Statistical Aggregation**
  - This process involves collecting and summarizing data to extract meaningful insights. For instance, if you have data from sensors, like `(sensorID, temperature)`, you can calculate the average temperature for each sensor. This helps in understanding the overall performance or behavior of the sensors.
  - *Map phase*: In this step, you gather the necessary data points. For example, you might collect the temperature readings and a count of 1 for each reading to help calculate averages later.
  - *Reduce phase*: Here, you perform calculations like sums, averages, or variances using the data collected in the map phase. This step is crucial for deriving insights from the raw data.
- **Log Analysis**
  - This involves examining log files to understand system behavior or identify issues. For example, by analyzing logs with entries like `(status code, 1)`, you can count how often each status code appears, which is useful for identifying errors or monitoring system health.
  - *Map phase*: During this phase, you parse the logs to extract important information such as timestamps, IP addresses, and status codes. You then emit key-value pairs that represent the metrics you want to analyze.
  - *Reduce phase*: This step involves aggregating the data, such as counting the number of errors or the number of hits from each IP address, to provide a clearer picture of system performance.

- **Implementation considerations**
  - These techniques are essential for tasks like monitoring system performance, setting up alerts for unusual activity, and analyzing trends over time.
  - They are particularly valuable in environments with large-scale logs from distributed systems, where manual analysis would be impractical. These methods allow for efficient processing and analysis of vast amounts of data.